

AI Calibration Project

Experimental Design, Provenance, Hallucination Tracking, and Traceability

Daniel M. Topa

August 6, 2026

Contents

Chapter 1

Purpose and Scope

The purpose of this project is to use a fixed calibration corpus to probe the behavior of an AI analysis pipeline as its training history, prompt configuration, input evidence, and inference settings change.

The central objective is not merely to determine whether training improves an aggregate score. The project is intended to determine whether training changes the model’s behavioral trajectory, whether those changes are reproducible, whether training order matters, whether prompts alter the character of the analysis, and whether every substantive conclusion can be traced back to objective evidence.

The project therefore treats AI behavior as an experimental subject. Each run is regarded as a controlled experiment whose complete input configuration is bound to its output through persistent identifiers, content hashes, provenance records, and immutable run metadata.

The calibration corpus is used as a probe. It should remain fixed and should ideally be excluded from all training activity. The same calibration cases are applied repeatedly to different model states, prompt versions, training sequences, and inference configurations. This makes it possible to characterize how the system changes over time and to isolate the sources of variation.

Chapter 2

Core Notation

It is useful to distinguish model states, training datasets, prompts, source documents, calibration cases, and inference settings.

2.1 Model State

Let

$$M_k$$

denote the model state after the k th training event or model modification.

The unmodified initial model is

$$M_0.$$

2.2 Training Datasets

Let

$$T_k$$

denote the k th training dataset.

The original notation

$$S_0 \cup S_1 \cup S_2$$

is not sufficient for expressing sequential training behavior because set union is commutative by definition:

$$S_0 \cup S_1 \cup S_2 = S_0 \cup S_2 \cup S_1.$$

The scientifically meaningful question is not whether the sets are equal, but whether the training process is order-independent.

2.3 Prompts

Let

$$P_0, P_1, P_2, \dots$$

denote prompt or instruction configurations.

A prompt configuration should include all system instructions, task instructions, rubrics, examples, formatting constraints, evidence requirements, and other text that may influence the model's response.

2.4 Input Documents and Data Sheets

Let

$$D_1, D_2, D_3, \dots$$

denote source documents, forms, data sheets, or other input evidence supplied to the pipeline.

2.5 Calibration Corpus

Let

$$C$$

denote the fixed calibration corpus.

The calibration corpus may contain multiple calibration cases:

$$C = \{C_1, C_2, \dots, C_n\}.$$

Each case should have known or adjudicated reference information, including expected findings, ratings, supporting evidence, and known ambiguities.

2.6 Inference Configuration

Let

$$\phi_r$$

denote the inference configuration associated with run r .

The inference configuration may include:

- temperature;
- random seed;
- top- p or other decoding parameters;
- retrieval settings;
- chunking method;
- context-window policy;
- model parameters;
- tool configuration;
- embedding model;
- vector-database configuration;
- evidence-ranking method;
- stopping criteria;
- output schema.

Chapter 3

Training as an Ordered Operation

3.1 Training Operator

Let

$$\mathcal{T}(M, T, P, \theta)$$

mean “apply training procedure $\mathcal{T}$ to model M , using training dataset T , prompt or instruction configuration P , and training parameters θ .”

Sequential training on T_1 followed by T_2 is:

$$M_{12} = \mathcal{T}(\mathcal{T}(M_0, T_1, P_1, \theta_1), T_2, P_2, \theta_2).$$

The reversed sequence is:

$$M_{21} = \mathcal{T}(\mathcal{T}(M_0, T_2, P_2, \theta_2), T_1, P_1, \theta_1).$$

The experimental question is:

$$M_{12} \stackrel{?}{\approx} M_{21}.$$

In general, one should expect:

$$M_{12} \neq M_{21}.$$

Training is path-dependent. The second dataset may reinforce, overwrite, distort, or partially erase behavior induced by the first dataset.

3.2 Behavioral Effects of Training Order

Training-order experiments can reveal:

- order dependence;
- catastrophic forgetting;
- positive transfer;
- negative transfer;
- interaction between datasets;
- sensitivity to prompt formulation;

- sensitivity to random initialization;
- sensitivity to sampling and decoding;
- recovery or degradation after repeated exposure.

3.3 Model States to Compare

For two training datasets, the following model states should be treated as distinct experiments:

$$M_0,$$

$$M_1 = \mathcal{T}(M_0, T_1),$$

$$M_2 = \mathcal{T}(M_0, T_2),$$

$$M_{12} = \mathcal{T}(\mathcal{T}(M_0, T_1), T_2),$$

$$M_{21} = \mathcal{T}(\mathcal{T}(M_0, T_2), T_1),$$

and

$$M_{1 \cup 2} = \mathcal{T}(M_0, T_1 \cup T_2).$$

The final three are not equivalent:

$$M_{12}, \quad M_{21}, \quad M_{1 \cup 2}.$$

They distinguish:

- sequential order effects;
- joint-training effects;
- simple dataset-content effects.

Repeated exposure may also be tested. For example,

$$M_{121} = \mathcal{T}(\mathcal{T}(\mathcal{T}(M_0, T_1), T_2), T_1).$$

This tests whether returning to T_1 restores earlier behavior, reinforces it, or causes further distortion.

3.4 Ordered Training Sequence

Let the ordered training sequence be

$$\mathcal{S} = (T_1, T_2, \dots, T_k).$$

The order is part of the experimental object. Therefore,

$$(T_1, T_2) \neq (T_2, T_1),$$

even though

$$T_1 \cup T_2 = T_2 \cup T_1.$$

This distinction is the conceptual center of the experimental design.

Chapter 4

Inference Runs

An inference run may be represented as

$$Y_r = \mathcal{I}(M_k, P_j, D_i, \phi_r),$$

where:

- M_k is the model state;
- P_j is the prompt configuration;
- D_i is the source document or data sheet;
- ϕ_r is the inference configuration;
- Y_r is the resulting output.

This separates:

- what the model has learned;
- what instructions it receives;
- what evidence it sees;
- how inference is performed.

Chapter 5

Calibration as a Behavioral Probe

5.1 Fixed Calibration Evaluation

For each model state M_k , evaluate the same calibration corpus:

$$E_k = \mathcal{E}(M_k, C, P_0).$$

The change from one model state to another may be represented as:

$$\Delta E_k = E_k - E_{k-1}.$$

A single aggregate score is insufficient. It can conceal important changes in behavior. Results should be retained at the level of model state, calibration case, and measured property:

$$E_{k,i,j},$$

where:

- k identifies the model state;
- i identifies the calibration case;
- j identifies the behavior being measured.

5.2 Behavioral Dimensions

The calibration process should measure multiple dimensions, including:

- rating correctness;
- evidence selection;
- factual support;
- traceability;
- omission rate;
- contradiction rate;
- unsupported-claim rate;
- calibration of confidence;
- stability across repeated runs;

- sensitivity to prompt wording;
- sensitivity to model choice;
- sensitivity to training order;
- human–machine agreement;
- consistency of attribution;
- consistency of extracted evidence;
- stability of generated themes.

The resulting set of measurements forms a behavioral fingerprint for each model state.

Chapter 6

Factorial Experimental Design

The complete system may be expressed as:

$$Y = f(M, T, P, D, R),$$

where:

- M is the initial model or model family;
- T is the ordered training sequence;
- P is the prompt configuration;
- D is the input document or calibration case;
- R is the replicated inference run.

A broader formulation is:

$$B = f(M_0, \mathcal{S}, P, D, \phi),$$

where B denotes observed behavior and $\mathcal{S}$ denotes the ordered training sequence.

A full experimental campaign may vary:

- initial model;
- model revision;
- training dataset;
- training order;
- joint versus sequential training;
- repeated exposure;
- prompt version;
- evidence supplied;
- retrieval strategy;
- chunking strategy;
- temperature;
- random seed;
- model context;

- output format;
- post-processing rules.

Chapter 7

Persistent Identity and Provenance

7.1 UUIDs

Universally unique identifiers should identify objects, not merely files.

Recommended identifiers include:

```
model_uuid
model_state_uuid
training_dataset_uuid
training_event_uuid
prompt_uuid
document_uuid
passage_uuid
calibration_case_uuid
run_uuid
output_uuid
claim_uuid
evidence_link_uuid
finding_uuid
assessment_uuid
review_uuid
reviewer_uuid
pipeline_version_uuid
environment_uuid
retrieval_config_uuid
```

The `run_uuid` should bind one complete execution configuration to its output.

For example:

```
run_uuid
  model_state_uuid
  prompt_uuid
  document_uuid
  calibration_case_uuid
  retrieval_config_uuid
  pipeline_version_uuid
```

```
environment_uuid
random_seed
decoding_parameters
timestamp
output_uuid
```

7.2 UUIDs and Content Hashes

A UUID establishes identity. A cryptographic hash establishes content integrity.

For example:

```
prompt_uuid
prompt_sha256
```

The UUID means:

This is the object identified as Prompt 17.

The hash means:

These are exactly the bytes used in Prompt 17.

Both should be stored. A UUID alone cannot prove that content has not been edited or accidentally replaced.

Recommended hashes include:

```
model_weights_sha256
training_dataset_sha256
prompt_sha256
document_sha256
pipeline_configuration_sha256
source_corpus_sha256
output_sha256
environment_manifest_sha256
```

7.3 Immutable Runs

Results should not be updated in place.

Each execution should create a new immutable run containing:

- a unique run UUID;
- source hashes;
- model-state UUID;
- prompt UUID;
- configuration hashes;
- software provenance;
- parent run, when applicable;

- timestamps;
- complete intermediate outputs;
- final outputs;
- evaluation results.

A run record may contain:

```
run_uuid: R-20260806-00017
parent_run_uuid: R-20260805-00009
change_reason: Revised duration-basis prompt
```

A separate pointer may identify the currently approved run:

```
/current/approved_run_uuid
```

This pointer should refer to an immutable run rather than replace or overwrite it.

Chapter 8

Model Lineage

Every trained model state should identify its parent and the training event that produced it.

```
model_state_uuid
parent_model_state_uuid
training_event_uuid
training_dataset_uuid
training_code_commit
training_configuration_hash
resulting_weights_hash
```

The resulting lineage forms a directed acyclic graph:

```
M0
|-- M1          trained on T1
|   '-- M12     then trained on T2
|
'-- M2          trained on T2
    '-- M21     then trained on T1
```

This allows queries such as:

- Which model states descend from M_0 through training dataset T_1 ?
- Which outputs were generated from a state whose most recent training event used T_2 ?
- Which model states share the same parent?
- Which training sequence produced the lowest unsupported-claim rate?
- Did a later training event restore or erase earlier behavior?

Chapter 9

Prompt Lineage

Prompts should be immutable, versioned objects with explicit lineage.

A prompt record may contain:

```
prompt_uuid
parent_prompt_uuid
prompt_text
prompt_hash
change_reason
author
created_at
```

Prompt lineage may be represented as:

```
P0
|-- P1: added evidence citation requirement
|-- P2: added confidence rubric
'-- P3: prohibited unsupported inference
```

Prompt experiments may reveal tradeoffs such as:

P_1 : lower unsupported claims, higher omission rate,

and

P_2 : better rating calibration, greater verbosity.

This converts prompt engineering from informal wording adjustment into an empirical discipline.

Chapter 10

Hallucination Tracking

10.1 Operational Definition

Hallucination should not be represented only as a single Boolean value.

A statement such as

```
hallucination = true
```

is too coarse to support diagnosis, comparison, or corrective action.

Each substantive output should be decomposed into atomic claims. Every claim should be classified according to its relationship to the available evidence.

A claim record may include:

```
claim_uuid  
output_uuid  
claim_text  
claim_type  
support_status  
supporting_passage_uuids  
contradicting_passage_uuids  
review_status  
reviewer_uuid
```

10.2 Support Categories

Recommended support categories include:

```
DIRECTLY_SUPPORTED  
SUPPORTED_BY_INFERENCE  
PARTIALLY_SUPPORTED  
UNSUPPORTED  
CONTRADICTED  
NOT_VERIFIABLE
```

These categories distinguish direct evidence from reasonable inference and separate unsupported material from material that is contradicted by the source.

10.3 Failure Taxonomy

The term hallucination includes several distinct failure modes.

10.3.1 Fabrication

The output introduces information that is absent from the evidence.

10.3.2 Contradiction

The output conflicts with the available evidence.

10.3.3 Overstatement

The evidence supports a limited statement, but the output strengthens it beyond what the source justifies.

For example, the evidence may support “partially developed,” while the output states “fully mature.”

10.3.4 Attribution Error

A statement belonging to one participant, product element, organization, or document is assigned to another.

10.3.5 Traceability Failure

A conclusion may be plausible, but the cited passage does not actually support it.

10.3.6 Unsupported Synthesis

Multiple facts may be present individually, but the model invents a relationship among them that the evidence does not establish.

10.3.7 Omission-Induced Distortion

The model omits qualifying, conflicting, or limiting evidence and thereby produces a misleading summary.

10.3.8 Entity Confusion

The model merges separate participants, systems, products, schedules, or review areas.

10.3.9 Temporal Confusion

The model attributes a fact, finding, or status to the wrong time period or model state.

10.3.10 False Precision

The model supplies unsupported numerical values, confidence levels, dates, rankings, or causal statements.

These failure classes should be tracked separately rather than collapsed into a single aggregate hallucination count.

Chapter 11

Claim-to-Evidence Traceability

The strongest traceability mechanism is a graph connecting source evidence to final assessments:

Document → Passage → Evidence Item → Claim → Finding → Assessment.

A traceability chain may include:

```
document_uuid: D-17
passage_uuid: P-416
claim_uuid: C-88
finding_uuid: F-12
assessment_uuid: A-3
```

Every generated assessment should include:

```
assessment_text
assessment_rating
supporting_claim_uuids
supporting_passage_uuids
prompt_uuid
model_state_uuid
run_uuid
```

A reviewer should be able to move backward from a conclusion such as

Cross-product integration confidence: Low

to:

- the generated assessment;
- the findings supporting the assessment;
- the atomic claims underlying those findings;
- the exact source passages;
- the source page;
- the original bounding box;
- the original document image;

- the prompt used;
- the model state;
- the complete run configuration.

This is audit-grade traceability.

Chapter 12

Quantitative Traceability Metrics

Traceability should be measured, not merely asserted.

12.1 Evidence Coverage

$$\text{Coverage} = \frac{\text{supported substantive claims}}{\text{all substantive claims}}.$$

12.2 Citation Precision

$$\text{Citation Precision} = \frac{\text{citations that genuinely support the claim}}{\text{all citations}}.$$

12.3 Evidence Recall

$$\text{Evidence Recall} = \frac{\text{relevant evidence cited}}{\text{relevant evidence available}}.$$

12.4 Attribution Accuracy

$$\text{Attribution Accuracy} = \frac{\text{correct entity assignments}}{\text{all entity assignments}}.$$

12.5 Unsupported-Claim Rate

$$H_u = \frac{\text{unsupported claims}}{\text{all claims}}.$$

12.6 Contradiction Rate

$$H_c = \frac{\text{contradicted claims}}{\text{all claims}}.$$

12.7 Omission Rate

$$H_o = \frac{\text{material reference facts omitted}}{\text{material reference facts available}}.$$

12.8 Traceability Completeness

$$T_c = \frac{\text{claims with complete source lineage}}{\text{all substantive claims}}.$$

The term hallucination rate should be used cautiously. A failure taxonomy is more informative because it reveals the nature and source of the error.

Chapter 13

Repeated Runs and Behavioral Stability

Even when the model state, prompt, and input document are fixed, repeated runs should be performed:

$$Y_{kji1}, Y_{kji2}, \dots, Y_{kjiR}.$$

Repeated runs reveal:

- whether ratings change;
- whether evidence citations change;
- whether atomic claims change;
- whether unsupported claims appear intermittently;
- whether contradictions are stable or sporadic;
- whether confidence tracks actual stability;
- whether the same conclusion is reached through different evidence;
- whether narrative wording is unstable while the underlying finding remains stable.

A representative stability report might be:

Rating agreement:	95%
Evidence-set agreement:	61%
Narrative similarity:	48%
Unsupported-claim rate:	3%

Such a result would indicate that the model reaches a stable rating while relying on a comparatively unstable set of evidence. That is more informative than simply stating that the outputs appear consistent.

13.1 Evidence-Set Similarity

Let E_a and E_b denote the sets of passages cited by two runs.

Their Jaccard similarity is:

$$J(E_a, E_b) = \frac{|E_a \cap E_b|}{|E_a \cup E_b|}.$$

This can be used to measure how consistently the model selects evidence.

13.2 Claim-Set Similarity

A similar measure can be applied to atomic claim sets:

$$J(C_a, C_b) = \frac{|C_a \cap C_b|}{|C_a \cup C_b|}.$$

13.3 Rating Stability

For categorical ratings, stability may be represented as the proportion of replicated runs producing the modal rating.

$$S_R = \frac{\text{number of runs producing the modal rating}}{\text{total replicated runs}}.$$

Chapter 14

Tracking Variation by Pipeline Stage

The pipeline should preserve intermediate products so that uncertainty can be localized.

Source Document → Extracted Text → Normalized Passages → Retrieved Evidence → Classified Findings → Rating

Variation should be tracked independently at each layer:

- extraction variation;
- normalization variation;
- chunking variation;
- retrieval variation;
- evidence-selection variation;
- claim-generation variation;
- classification variation;
- rating variation;
- narrative variation.

This permits investigation of questions such as:

- Is instability introduced during OCR?
- Does chunking alter which evidence is retrieved?
- Does retrieval stabilize while narrative generation remains variable?
- Does a model produce the same rating from different evidence?
- Does a prompt improve traceability while reducing evidence recall?
- Does training reduce unsupported claims but increase omissions?

Chapter 15

HDF5 as the Experimental Store

HDF5 is well suited to storing dense analytical products, repeated-run data, embeddings, probability vectors, and claim–evidence matrices.

A possible organization is:

```
/experiments/{experiment_uuid}/  
  model_states  
  training_sequences  
  prompts  
  documents  
  calibration_cases  
  runs  
  ratings  
  confidence_vectors  
  embeddings  
  claim_support_matrices  
  evidence_selection_matrices  
  repeated_run_metrics
```

15.1 Claim-Support Matrix

A claim-support matrix may be defined as:

$$A_{r,c} = \begin{cases} 1, & \text{if run } r \text{ supports claim } c, \\ 0, & \text{otherwise.} \end{cases}$$

15.2 Evidence-Selection Matrix

An evidence-selection matrix may be defined as:

$$E_{r,p} = \begin{cases} 1, & \text{if run } r \text{ cites passage } p, \\ 0, & \text{otherwise.} \end{cases}$$

These matrices allow direct comparison of evidence selection, claim generation, and support patterns across runs.

15.3 Recommended HDF5 Content

HDF5 is particularly appropriate for:

- embeddings;
- token-level scores;
- probability vectors;
- repeated-run numerical results;
- confidence vectors;
- large structured payloads;
- evidence-selection matrices;
- claim-support matrices;
- model comparison arrays;
- calibration response tensors.

Chapter 16

Hybrid Storage Architecture

HDF5 should not be the sole storage mechanism.

A relational catalog is preferable for UUID relationships, metadata queries, lineage searches, and joins. A practical architecture is:

Raw documents

- Original DOCX/PDF/image files

SQLite or DuckDB catalog

- Participants

- Documents

- Passages

- Calibration cases

- Runs

- Relationships

- Ratings

- Findings index

- Claims

- Reviews

- Provenance summary

HDF5 analytical store

- Extracted arrays

- Embeddings

- Token-level outputs

- Model scores

- Repeated-run results

- Large structured payloads

JSON sidecars

- Human-readable run manifests

- Configuration

- Prompt templates

- Schema definitions

A useful division of responsibility is shown in Table ??.

Table 16.1: Recommended storage by information type

Information	Preferred Storage
Original forms	Native files
Stable identifiers and relationships	SQLite or DuckDB
Extracted text passages	SQLite, Parquet, or both
Embeddings	HDF5
Token probabilities and dense arrays	HDF5
Ratings and findings	SQLite or DuckDB
Repeated-run numerical results	HDF5
Prompt text and configuration	JSON plus hashes
Human review decisions	SQLite or DuckDB
Large run snapshots	HDF5

Chapter 17

File Organization

A single enormous HDF5 file should be avoided initially.

A safer organization is:

```
corpus/  
    catalog.sqlite  
  
runs/  
    2026/  
        08/  
            run-R000001.h5  
            run-R000002.h5  
            run-R000003.h5  
  
manifests/  
    run-R000001.json  
    run-R000002.json
```

One HDF5 file per run, batch, or experimental campaign provides:

- simpler recovery;
- easier archival;
- reduced write contention;
- clear immutability;
- easier movement between systems;
- reduced risk from corruption.

The relational catalog records where each run file is located.

Chapter 18

Participant and Entity Tracking

Participants, reviewers, contractors, organizations, and product elements should be represented through stable identifiers.

A participant record may include:

```
participant_uuid  
organization_uuid  
role_code  
product_element_uuid  
function_code  
review_session_uuid
```

Personally identifying information should not be repeated throughout the analytical datasets. Instead of storing a person's name in every result, use:

```
participant_uuid = PERSON-0042
```

Any mapping between identifiers and personal information should be maintained separately with stricter access control.

HDF5 should not be treated as an access-control system. Once a user can read the file, internal groups do not provide meaningful row-level confidentiality.

Chapter 19

Questions the Project Can Answer

The proposed architecture supports questions that ordinary document storage cannot answer cleanly.

- How stable is the confidence rating across prompt formulations?
- Which participants produce the greatest analytical uncertainty?
- Which assessment areas are most model-sensitive?
- Does evidence selection stabilize before narrative wording stabilizes?
- Does a model change alter conclusions or merely prose?
- Which human reviewers systematically disagree with the pipeline?
- Are low-confidence assessments driven by missing evidence or conflicting evidence?
- How much variation is introduced by OCR, classification, retrieval, and generation separately?
- Does training order alter the model’s findings?
- Does joint training behave differently from sequential training?
- Does repeated exposure restore earlier behavior or amplify later behavior?
- Which prompt version minimizes unsupported claims?
- Which prompt version maximizes evidence recall?
- Are citations stable even when the narrative changes?
- Are ratings stable when the supporting evidence is unstable?
- Which model state has the best human–machine agreement?
- Which model state provides the strongest claim-to-evidence traceability?

Chapter 20

Recommended Experimental Workflow

A disciplined workflow may proceed as follows.

1. Establish a fixed calibration corpus.
2. Define reference findings, ratings, evidence, and known ambiguities.
3. Assign persistent UUIDs to every model, dataset, prompt, document, passage, claim, run, and review.
4. Hash all content-bearing objects.
5. Define an initial model state M_0 .
6. Define training datasets $T_1, T_2, \dots, T_k$.
7. Define prompt configurations $P_0, P_1, \dots, P_j$.
8. Define the ordered training sequences to be tested.
9. Train each model state while preserving complete lineage.
10. Run the fixed calibration corpus against every model state.
11. Replicate inference runs under identical configurations.
12. Preserve every intermediate pipeline output.
13. Decompose generated narratives into atomic claims.
14. Link each claim to supporting and contradicting passages.
15. Classify each claim by support status.
16. Perform human adjudication.
17. Compute behavioral, traceability, hallucination, and stability metrics.
18. Compare model states, prompt versions, and training sequences.
19. Preserve all outputs as immutable experimental records.

Chapter 21

Central Research Question

The project can be summarized by the following question:

Given a fixed calibration corpus, how does model behavior change as a function of training history, prompt history, evidence presented, and inference configuration?

Symbolically:

$$B = f(M_0, \mathcal{S}, P, D, \phi).$$

The project is not merely an exercise in improving an AI score. It is an investigation of how training changes behavior, whether those changes are reproducible, whether training order matters, whether prompt changes alter the model’s analytical character, and whether every conclusion can be traced to evidence.

This framing turns an AI demonstration into a defensible experimental program.